



Universidad Autónoma de Nuevo
León

Facultad de Ingeniería Mecánica y Eléctrica

Práctica 4

Árboles binarios y Árboles de decisión

Nombre	Ranfery Josua Peregrina Morales
Matrícula	1924910
Grupo	000
Carrera	ITS
Asignatura	Tópicos Selectos de Ciencias de la Ing. III
Docente	Raymundo Said Zamora Pequeño
Fecha	22/3/26

Periodo Enero – Junio 2026

Índice

Objetivo	3
Marco teórico	3
Minería de datos	3
Clasificación supervisada.....	3
Árboles de decisión	3
Entropía en teoría de la información	4
Ganancia de información	4
Algoritmos ID3 y C4.5	5
Aplicación de los árboles de decisión en minería de datos.....	5
Justificación del uso de logaritmos en la entropía	5
Desarrollo	6
Parte 1: Árboles de decisión en Excel.....	6
Parte 2: Árboles binarios en R	6
Experimentación con el script de <i>ctree</i>	7
Experimentación con el script de <i>rpart</i>	8
Cálculos y resultados	9
Resultados de Árboles de decisión	9
Conclusión del análisis del ejercicio de Árboles de decisión.....	12
Resultados de Árboles binarios	13
CTree.....	13
Conclusión del análisis del ejercicio de CTree	18
Rpart.....	18
Conclusión del análisis del ejercicio de Rpart	25
Conclusiones generales	25
Referencias	26

Objetivo

Encontrar el árbol de decisión de mínima entropía para un registro de datos categóricos.

Marco teórico

Minería de datos

La **minería de datos** es una disciplina dentro del área de ciencia de datos y aprendizaje automático que se enfoca en la extracción de patrones, relaciones y conocimiento útil a partir de grandes volúmenes de datos[2]. Su objetivo principal es transformar datos sin procesar en información significativa que pueda ser utilizada para la toma de decisiones, predicción y descubrimiento de tendencias ocultas.

El proceso de minería de datos forma parte de un flujo más amplio conocido como **KDD (Knowledge Discovery in Databases)**, el cual incluye etapas como limpieza de datos, integración, selección, transformación, minería y evaluación de los patrones obtenidos.

Dentro de las técnicas de minería de datos se encuentran:

- Clasificación
- Agrupamiento (clustering)
- Reglas de asociación
- Análisis de anomalías
- Árboles de decisión

Clasificación supervisada

La **clasificación supervisada** es una técnica de aprendizaje automático en la cual un modelo es entrenado utilizando datos previamente etiquetados para poder predecir la clase o categoría de nuevas observaciones. En este contexto, los árboles de decisión son una de las herramientas más utilizadas debido a su interpretabilidad y facilidad de implementación.

Árboles de decisión

Un **árbol de decisión** es una estructura jerárquica utilizada para representar decisiones y sus posibles consecuencias. Está compuesto por:

- **Nodo raíz:** representa el conjunto completo de datos.
- **Nodos internos:** representan pruebas sobre atributos.
- **Ramas:** representan los resultados de las pruebas.
- **Hojas:** representan la decisión o clase final.

Los árboles de decisión se construyen dividiendo recursivamente un conjunto de datos en subconjuntos más pequeños, de tal manera que cada división reduzca la incertidumbre sobre la variable objetivo.

Los algoritmos más conocidos para la generación de árboles de decisión son:

- ID3
- C4.5
- CART

El algoritmo **ID3 (Iterative Dichotomiser 3)**, desarrollado por Ross Quinlan[1], utiliza la entropía y la ganancia de información como criterios para seleccionar el mejor atributo en cada división del árbol.

Entropía en teoría de la información

La **entropía** es un concepto introducido por Claude Shannon en la teoría de la información para medir la incertidumbre o desorden presente en una variable aleatoria. En minería de datos, la entropía se utiliza para evaluar qué tan mezcladas o heterogéneas están las clases dentro de un conjunto de datos[5].

La fórmula general de la entropía es:

$$H(X) = - \sum_{i=1}^n p_i \log(p_i)$$

Donde:

- p_i es la probabilidad de ocurrencia del evento i
- n es el número total de eventos posibles

Si todos los elementos pertenecen a la misma clase, la entropía es 0, lo que indica que no hay incertidumbre. Si las clases están perfectamente balanceadas, la entropía alcanza su valor máximo, indicando máxima incertidumbre.

Entropía en árboles de decisión

En el contexto de los árboles de decisión, la entropía se utiliza para medir la pureza de un nodo. Un nodo con baja entropía contiene datos más homogéneos, lo cual es deseable, ya que facilita la clasificación.

Durante la construcción de un árbol, el objetivo es seleccionar el atributo que reduzca lo más posible la entropía en los nodos hijos. Este proceso se conoce como **minimización de la entropía** o, de manera equivalente, **maximización de la ganancia de información**. [3]

Ganancia de información

La **ganancia de información** es una métrica que mide la reducción de entropía después de dividir un conjunto de datos utilizando un atributo específico. Se calcula como la diferencia entre la entropía original y la entropía ponderada de los subconjuntos resultantes.

$$IG(S, A) = H(S) - \sum_{i=1}^k \left(\frac{|S_i|}{|S|} \right) H(S_i)$$

- S es el conjunto de datos original
- A es el atributo evaluado
- S_i son los subconjuntos generados al dividir por A

El atributo con mayor ganancia de información es seleccionado para dividir el nodo en los algoritmos clásicos como ID3.

Algoritmos ID3 y C4.5

El algoritmo **ID3** construye árboles de decisión de manera recursiva, seleccionando en cada paso el atributo que produce la mayor ganancia de información[1]. Este proceso continúa hasta que:

- todos los ejemplos pertenecen a la misma clase
- no quedan atributos por evaluar

Posteriormente, el algoritmo **C4.5** fue desarrollado como una mejora del ID3[4], introduciendo características como:

- manejo de atributos numéricos
- tratamiento de datos faltantes
- poda del árbol
- uso de la **razón de ganancia** para evitar sesgos hacia atributos con muchos valores distintos

Aplicación de los árboles de decisión en minería de datos

Los árboles de decisión son ampliamente utilizados en:

- diagnóstico médico
- sistemas de recomendación
- análisis de riesgo financiero
- clasificación de documentos
- análisis de comportamiento del consumidor

Su popularidad se debe a que generan modelos interpretables, fáciles de visualizar y capaces de manejar datos categóricos y numéricos.

Justificación del uso de logaritmos en la entropía

El uso del logaritmo en la fórmula de entropía proviene de la **teoría de la información**, donde se interpreta como la cantidad de información necesaria para codificar un evento. Dependiendo de la base del logaritmo, la entropía puede medirse en bits (base 2), nats (base e) o bans (base 10). En esta práctica se utiliza logaritmo base 10 para simplificar los cálculos manuales, sin afectar la comparación relativa entre árboles.

Desarrollo

Parte 1: Árboles de decisión en Excel

En esta práctica, en lugar de aplicar directamente algoritmos automáticos como ID3, se analizan manualmente todas las combinaciones posibles de atributos para construir árboles de decisión y calcular la entropía en cada nivel. Este enfoque permite comprender de forma más profunda cómo la elección del orden de los atributos afecta la incertidumbre total del sistema.

En esta práctica se nos fue presentado un DataSet que guardaba registros para tres variables:

Make (M) en donde se enlistan las marcas de los coches. Electric Vehicle Type (T) en donde se enlistan el tipo de coche eléctrico que es. Clean Alternative Fuel Vehicle (CAFV) Eligibility (E) que es la elegibilidad.

Dado un conjunto de atributos M, T, E , es posible generar todas las permutaciones:

1. MTE
2. MET
3. EMT
4. ETM
5. TME
6. TEM

Para cada una de estas permutaciones se construyen árboles de decisión en los cuales:

1. Se agrupan los datos según el primer atributo.
2. Posteriormente se subdividen según el segundo atributo.
3. Finalmente se subdividen según el tercer atributo.

La entropía se calcula en cada nivel utilizando las frecuencias relativas de cada combinación, lo que equivale a calcular la entropía conjunta de las variables involucradas.

Un árbol con menor entropía total representa un modelo más determinista, es decir, uno en el que las decisiones pueden tomarse con mayor certeza y menor ambigüedad.

Para esta práctica, dado el dataset con registros para las tres variables, se desarrollaron árboles de decisión acomodando por filas las distintas variables en diferente combinación de orden, se calculó su frecuencia absoluta (Según cada registro cumpliera con las tres características), se determinó su frecuencia relativa y finalmente su entropía.

Para cada árbol, se calculó la entropía de cada nivel, tres niveles por árbol, pues en cada nivel se calculó la entropía de usar una estructura con “n” variables (Del 1 al 3), donde n es el nivel.

Parte 2: Árboles binarios en R

En clase se nos impartió un ejemplo de práctica sobre **árboles de decisión** con R, con modelos predictivos. De dicho ejemplo se encargó para la práctica del laboratorio experimentar con algunos parámetros y a continuación presentaremos en 2 subdivisiones dicha experimentación.

1. Cambios experimentales el script de clase.
2. Aplicación con un dataset distinto (*BR.csv*)

Experimentación con el script de *ctree*

En clase se presentó el siguiente script:

```
library(datasets)
library(caTools)
library(party)
library(dplyr)
library(magrittr)

data("readingsSkills")
head(readingsSkills)
nativeSpeaker age shoeSize score
yes 5 24.83189 32.29385
yes 6 25.95238 36.63105
no 11 30.42170 49.60593
yes 7 28.66450 40.28456
yes 11 31.88207 55.46085
yes 10 30.07843 52.83124

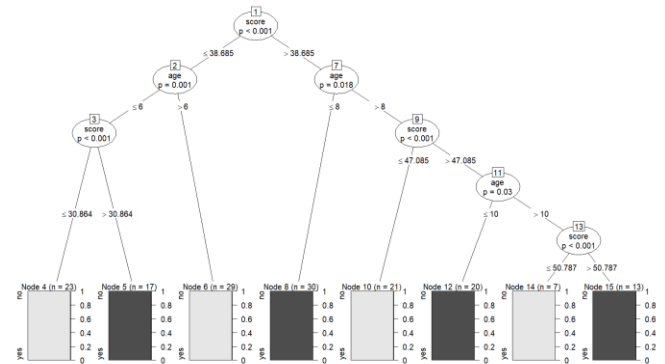
sample_data <- sample.split(readingsSkills$nativeSpeaker, SplitRatio = 0.8)
train_data <- subset(readingsSkills, sample_data == TRUE)
test_data <- subset(readingsSkills, sample_data == FALSE)

model <- ctree(nativeSpeaker ~ ., data = train_data)
plot(model)

predict_model <- predict(model, test_data)
m_at <- table(test_data$nativeSpeaker, predict_model)
m_at
  predict_model
no yes
no 18 2
yes 0 20

ac_Test <- sum(diag(m_at)) / sum(m_at)
print(paste('Precisión', ac_Test))
[1] "Precisión 0.95"

new_data <- data.frame(
  age = as.integer(c(7, 8)),
  shoeSize = as.numeric(c(35, 37)),
  score = as.numeric(c(22, 30))
)
```



Este árbol intenta predecir si alguien es un "hablante nativo" (*nativeSpeaker*) basándose en su edad, tamaño de zapato y puntuación.

```
> pred_new <- predict(model, new_data)
> pred_new
[1] no no
Levels: no yes
```

Y para la práctica se nos fue requerido cambiar y comparar algunos parámetros, en este caso se hicieron cambios en el parámetro *SplitRatio*, que actualmente inicia con el valor de 0.8, es decir 80% de los datos de entrenamiento.

Probaremos con 0.6 (dejando solo el 60% para aprender) y con 0.9 (El modelo usa casi todo para aprender).

En la parte final del script, donde creamos el *data.frame*, se cambiarán los valores de *age*, *shoeSize* o *score*. En este caso, la elección fue alguien de 12 años con una puntuación muy baja para ver qué predice el árbol.

Con esto buscamos hacer 2 observaciones:

- **La Precisión (*ac_Test*):** Si se baja el *SplitRatio* a 0.6, es probable que la precisión baje porque el modelo no tuvo suficientes ejemplos para aprender las reglas.
- **El gráfico:** Al cambiar el ratio, hemos de observar si el árbol se vuelve más "alto" o más "pequeño". Esperaríamos comprobar que un árbol más alto tiene reglas más específicas.

Este experimento será hallado más adelante en el documento, en la sección de **Cálculos y resultados**.

Experimentación con el script de *rpart*

Este es un modelo de **regresión** (la respuesta es un número: el precio).

```
library(rpart)
library(rpart.plot)
library(caTools)
library(Metrics)

datos <- read.csv(file.choose(), stringsAsFactors = FALSE)

formula_modelo <- precio ~ .
muestra <- sample.split(datos$precio, splitRatio = 0.8)
train <- subset(datos, muestra)
test <- subset(datos, !muestra)

modelo_arbol <- rpart(
  formula_modelo,
  data = train,
  method = "anova",
  control = rpart.control(
    minsplit = 20,
    cp = 0.001
  )
)

cat("\n=== Tabla de complejidad (printcp) ===\n")
== Tabla de complejidad (printcp) ==
printcp(modelo_arbol)

egression tree:
part(formula = formula_modelo, data = train, method = "anova",
      control = rpart.control(minsplit = 20, cp = 0.001))

variables actually used in tree construction:
1] ano_construccion codigo_postal codigo_vista latitud
5] longitud m2_construccion m2_terreno pisos

oot node error: 8.2076e+13/800 = 1.026e+11

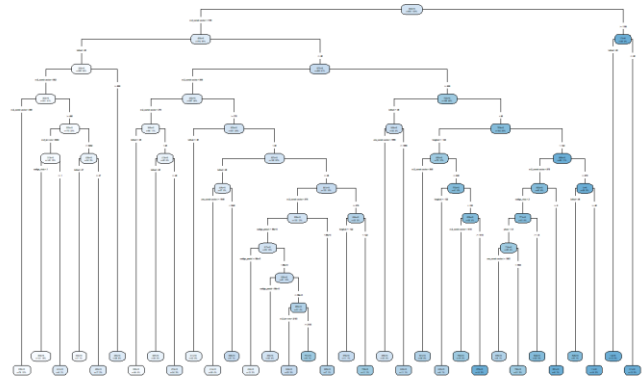
= 800

      CP nsplit rel error xerror xstd
0.3534560 0 1.00000 1.00460 0.172179
0.1382658 1 0.64654 0.76558 0.120516
0.0935647 2 0.50828 0.62464 0.107859
0.0742632 3 0.41471 0.51380 0.086559
0.0208358 4 0.34045 0.46212 0.078196
0.0184921 5 0.31961 0.41529 0.059939
0.0136375 6 0.30112 0.41333 0.059788
0.0118602 7 0.28748 0.40413 0.059693
0.0095224 8 0.27562 0.39108 0.059316
0 0.0086632 9 0.26610 0.38470 0.059134
1 0.0070912 10 0.25744 0.38492 0.059476
2 0.0070269 11 0.25035 0.37996 0.059437

> cp_opt <- modelo_arbol$cpstable[which.min(modelo_arbol$cpstable[, "xerror"]), "cp"]
> cat("\nMejor cp sugerido:", cp_opt, "\n")

Mejor cp sugerido: 0.001108086
>
> modelo_podado <- prune(modelo_arbol, cp = cp_opt)
>
> rpart.plot(modelo_podado, type = 4, extra = 101)
>
> pred_test <- predict(modelo_podado, newdata = test)
>
> RMSE <- sqrt(mean((test$precio - pred_test)^2))
> MAE <- mean(abs(test$precio - pred_test))
>
> SST <- sum( (test$precio - mean(test$precio))^2 )
> SSE <- sum( (test$precio - pred_test)^2 )
> R2 <- 1 - SSE/SST
>
> cat(sprintf("\n=== Métricas en TEST ===\nRMSE: %.4f\nMAE: %.4f\nR²: %.4f\n", RMSE, MAE, R2))

=== Métricas en TEST ===
RMSE: 221259.1372
MAE: 121811.2926
R²: 0.6190
>
>
> datos$pred_precio <- predict(modelo_podado, newdata = datos)
```



Cambiaremos 2 variables.

minsplit (en rpart.control):

Si se pone en 5, el árbol intentará crear ramas incluso para grupos muy pequeños de casas. Esto hace el árbol muy complejo.

Si se pone en 100, el árbol será muy simple porque solo creará ramas si hay al menos 100 casas que compartan una característica.

cp (Complexity Parameter):

Es el valor de "poda". Un cp de 0.001 (el que usamos en la clase) es muy bajo, lo que permite que el árbol crezca mucho.

Nosotros optaremos por subirlo a 0.1. Esperamos ver que el árbol se vuelve mucho más pequeño. El objetivo es encontrar el cp que dé el menor error sin ser un árbol infinito.

Este experimento será hallado más adelante en el documento, en la sección de **Cálculos y resultados.**

Cálculos y resultados

Resultados de Árboles de decisión

Para las permutaciones no tuve que realizar ningún algoritmo complicado de combinatoria, de hecho, los deducimos durante la hora de clase, al sólo ser 6 posibles combinaciones no fue difícil determinar las estructuras "molde" de cada árbol. Sin embargo, para el Script de Python que hice para automatizar el cálculo *Obtener-EntropiaArboles.py*, usé la librería *itertools*.

```
# ===== 4. Generar todas las permutaciones =====  
variables = ["M", "T", "E"]  
orders = list(itertools.permutations(variables))
```

De manera independiente, para tener una confirmación visual de los cálculos decidí organizarlos en filas y columnas de Excel, que es donde inicialmente estaba trabajando.

```
# Puse esto para que haga un Excel a parte.  
writer = pd.ExcelWriter("arboles_entropia.xlsx", engine="xlsxwriter")
```

Para cada nodo del árbol se calcularon las frecuencias absolutas y relativas de las combinaciones de variables presentes en el conjunto de datos. A partir de estas probabilidades se aplicó la fórmula de entropía de Shannon:

$$H(X) = \sum_{i=1}^n p_i \log(p_i)$$

[illegible][illegible][illegible]

[illegible][illegible]

E	T	M	FreqABS	FreqRel	Entropy
Clean Alte	Battery Elk AUD		1	0.01	0.02
Clean Alte	Battery Elk BMW		1	0.01	0.02
Clean Alte	Battery Elk CHEROLEE		3	0.03	0.045686
Clean Alte	Battery Elk FIAT		2	0.02	0.033979
Clean Alte	Battery Elk KIA		7	0.07	0.080843
Clean Alte	Battery Elk NISSAN		13	0.13	0.115817
Clean Alte	Battery Elk POLESTAR		1	0.01	0.02
Clean Alte	Battery Elk PORSCHE		1	0.01	0.02
Clean Alte	Battery Elk TESLA		26	0.26	0.152107
Clean Alte	Plug-in Hyl BMW		7	0.07	0.080843
Clean Alte	Plug-in Hyl CADILLAC		1	0.01	0.02
Clean Alte	Plug-in Hyl CHEROLEE		2	0.02	0.033979
Clean Alte	Plug-in Hyl HYUNDAI		1	0.01	0.02
Clean Alte	Plug-in Hyl KIA		3	0.03	0.045686
Clean Alte	Plug-in Hyl VOLVO		2	0.02	0.033979
Eligibility u	Battery Elk BMW		2	0.02	0.033979
Eligibility u	Battery Elk FORD		1	0.01	0.02
Eligibility u	Battery Elk TESLA		4	0.04	0.055918
Eligibility u	Battery Elk VOLKSWAGEN		1	0.01	0.02
Eligibility u	Battery Elk VOLVO		1	0.01	0.02
Not eligibl	Plug-in Hyl AUD		3	0.03	0.045686
Not eligibl	Plug-in Hyl BMW		6	0.06	0.073311
Not eligibl	Plug-in Hyl FORD		1	0.01	0.02
Not eligibl	Plug-in Hyl JEEP		4	0.04	0.055918
Not eligibl	Plug-in Hyl KIA		3	0.03	0.045686
Not eligibl	Plug-in Hyl VOLVO		3	0.03	0.045686
					1.178476

2.0224

Conclusión del análisis del ejercicio de Árboles de decisión

Los resultados muestran que el orden en que se seleccionan las variables influye significativamente en la cantidad de incertidumbre presente en el árbol de decisión. El árbol con menor entropía total fue **TEM**, lo que indica que comenzar la partición con la variable T produce la mayor reducción de incertidumbre en el conjunto de datos.

Esto sugiere que la variable T posee el mayor poder de discriminación, ya que permite separar los datos en grupos más homogéneos desde la primera división. En contraste, los árboles que comienzan con la variable M presentaron los valores de entropía más altos, lo que indica que esta variable no genera divisiones eficientes en los niveles iniciales y requiere el uso de más variables para reducir la incertidumbre.

La diferencia entre la entropía mínima y máxima obtenida (1.9663 frente a 3.2354) evidencia que la selección adecuada de la variable raíz puede reducir la incertidumbre del modelo en más del 35%, lo cual tiene un impacto directo en la calidad de la clasificación.

Resultados de Árboles binarios

CTree

En el programa original tenemos un modelo creado con estos valores de entrenamiento

```
sample_data <- sample.split(readingSkills$nativeSpeaker, SplitRatio = 0.8)
train_data <- subset(readingSkills, sample_data == TRUE)
test_data <- subset(readingSkills, sample_data == FALSE)
```

- **80% entrenamiento**
- **20% prueba**

Hay que cambiar el split para ver cómo afecta.

Probaremos con estos datos:

SplitRatio = 0.6

SplitRatio = 0.9

Cambiando el SplitRatio a 0.6

Los datos iniciales son los mismos, aún así se considera para el efecto que el modelo puede introducir cierta aleatoriedad durante el entrenamiento. Este comportamiento aleatorio que proviene de la función `sample.split()` que realiza un muestreo aleatorio para dividir los datos en entrenamiento y prueba.

Cada vez que se ejecuta `sample.split()`, se genera una división diferente de los datos, lo que produce distintos conjuntos de entrenamiento y, por lo tanto, diferentes árboles de decisión.

Así que hemos decidido fijar una semilla para hacer una comparación más precisa en donde no consideremos como factor de “ruido” la aleatoriedad. La semilla es mi *matrícula*.

```
# Fijar la semilla para reproducibilidad
set.seed(1924910) # Mi matrícula 🏆
```

```
data("readingSkills")
head(readingSkills)
nativeSpeaker age shoeSize score
yes 5 24.83189 32.29385
yes 6 25.95238 36.63105
no 11 30.42170 49.60593
yes 7 28.66450 40.28456
yes 11 31.88207 55.46085
yes 10 30.07843 52.83124
```

Aquí es donde tenemos el cambio de valores en nuestras variables experimentales. Vamos a fijar el *SplitRatio* a **0.6**, es decir, usar de nuestra muestra de datos, utilizar el 60% para entrenamiento del modelo. Ahora hay menos datos que se utilizan para entrenar el modelo.

```
sample_data <- sample.split(readingSkills$nativeSpeaker, SplitRatio = 0.6)
train_data <- subset(readingSkills, sample_data == TRUE)
test_data <- subset(readingSkills, sample_data == FALSE)
```

Ahora con esos datos inicializamos un modelo de árbol *ctree* y creamos el diagrama, así como la matriz de confusión. En donde observamos que aproximadamente un tercio de los intentos de predicción falló en acertar en “No” y tuvo menos errores para acertar el “Sí”.

```
model <- ctree(nativespeaker ~ ., data = train_data)
```

```
plot(model)
```

```
predict_model <- predict(model, test_data)
```

```
m_at <- table(test_data$nativespeaker, predict_model)
```

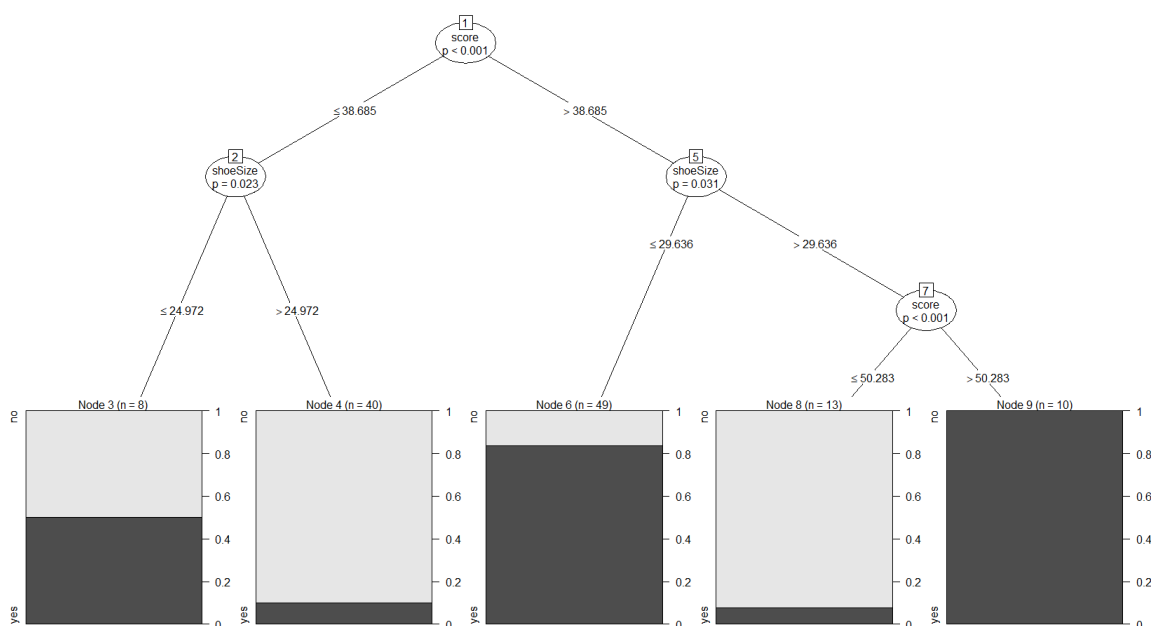
```
m_at
```

```
predict_model
```

```
no yes
```

```
no 30 10
```

```
yes 17 23
```



Como tuvo pocos datos para estudiar, el modelo no encontró suficiente evidencia estadística para hacer reglas complejas (por eso el árbol tiene menos nodos). Se quedó con reglas muy generales.

Ahora medimos la precisión del modelo entrenado, dividiendo la cantidad de veces que acertó entre la cantidad de veces que no acertó y nos indica un formidable 66%

Aunque es un valor por encima del 50%, para predicciones no es suficientemente estable.

```
> ac_Test <- sum(diag(m_at)) / sum(m_at)
```

```
> print(paste('Precisión', ac_Test))
```

```
[1] "Precisión 0.6625"
```

Después de probar el modelo con los datos de prueba, pasamos a hacer un nuevo frame para predicción en donde se le indican dos casos, de niños con edades de 7 y 8 años y el resto de los

datos preestablecidos para el ejemplo de clase. Su predicción es que ninguno de los dos es hablante nativo según el tamaño de sus zapatos y su puntaje *score*.

```
> new_data <- data.frame(
+   age = as.integer(c(7, 8)),
+   shoeSize = as.numeric(c(35, 37)),
+   score = as.numeric(c(22, 30))
+ )
>
> pred_new <- predict(model, new_data)
> pred_new
[1] no no
Levels: no yes
```

Cambiando el SplitRatio a 0.9

Los datos iniciales son los mismos, y para asegurar la replicabilidad del experimento y asegurarnos de que el cambio de resultados se produce específicamente por nuestro cambio de variables, usamos la misma semilla de generación para eliminar el factor de aleatoriedad.

```
# Fijar la semilla para reproducibilidad
set.seed(1924910) # Mi matrícula 🏆
data("readingskills")
head(readingskills)
nativeSpeaker age shoeSize score
yes 5 24.83189 32.29385
yes 6 25.95238 36.63105
no 11 30.42170 49.60593
yes 7 28.66450 40.28456
yes 11 31.88207 55.46085
yes 10 30.07843 52.83124
```

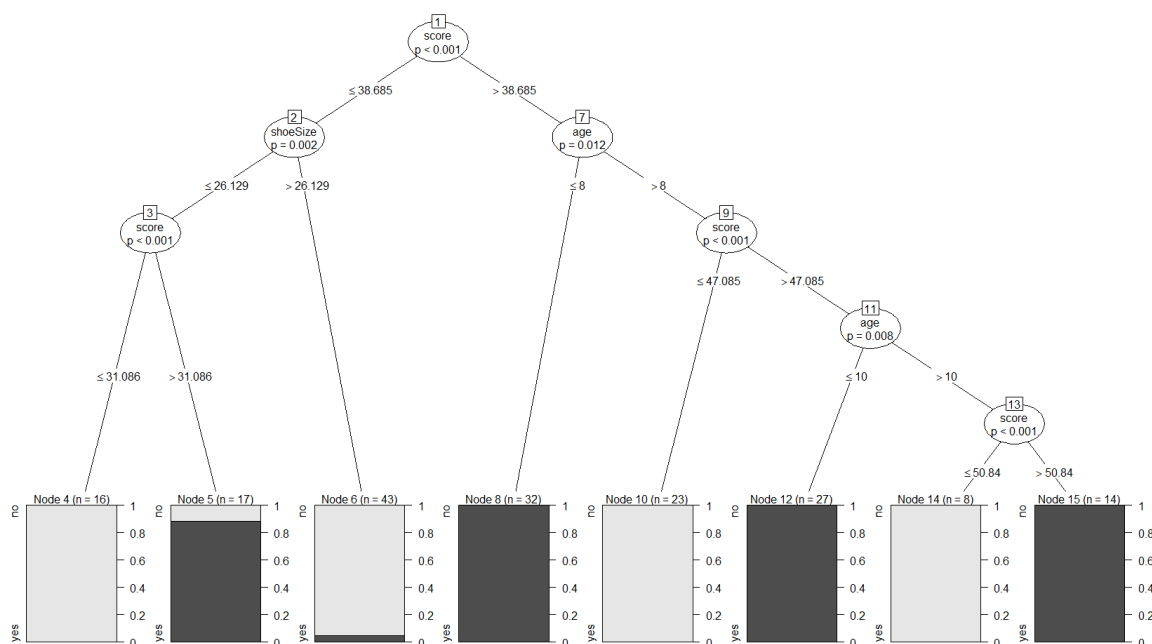
Después hacemos el cambio de variable. En el *SplitRatio* esta vez asignamos el 90% de los datos al entrenamiento del modelo, casi todos. Usaremos sólo el 10% de los casos para comparar resultados y dedicamos la gran mayoría al entrenamiento.

```
sample_data <- sample.split(readingskills$nativeSpeaker, splitRatio = 0.9)
train_data <- subset(readingskills, sample_data == TRUE)
test_data <- subset(readingskills, sample_data == FALSE)
```

Observamos que el cambio en la precisión es muy evidente, obteniendo un 90% de precisión, es decir, acertando en 9 de 10 casos. Los números de la matriz de confusión además de señalar una clara tendencia a las predicciones correctas, muestra también valores más bajos porque se usaron menos casos para probar el modelo. En el caso anterior eran números entre el 10 y el 30, ahora tenemos menos de 9.

```
model <- ctree(nativeSpeaker ~ ., data = train_data)
plot(model)
predict_model <- predict(model, test_data)
m_at <- table(test_data$nativeSpeaker, predict_model)
m_at
      predict_model
      no yes
no    9    1
yes   1    9
```

En esta matriz de confusión obtuvimos resultado completamente distintos. Fallando sólo en un caso. Aunque, también es que eran muy pocos datos para probar. Quizá si le poníamos más datos de prueba se equivocaba más veces.



Como tuvo tantos datos, creó un árbol con muchísimos nodos. Se entiende que incluso se "memorizó" los datos de entrenamiento creando reglas súper específicas.

```
ac_Test <- sum(diag(m_at)) / sum(m_at)
print(paste('Precisión', ac_Test))
1] "Precisión 0.9"
```

Una precisión muy alta, pero hay que considerar el tamaño del muestreo de *testeo*. Se le dio el 90% de los datos para estudiar. Sacar 90% en 20 preguntas es fácil, un solo error te baja 5 puntos. Esto se llama **Sobreajuste (Overfitting)**. El modelo parece perfecto, pero probablemente fallaría mucho porque sus reglas son demasiado rígidas y específicas para los datos que ya vio.

```
> new_data <- data.frame(
+ age = as.integer(c(7, 8)),
+ shoeSize = as.numeric(c(35, 37)),
+ score = as.numeric(c(22, 30))
+ )
> pred_new <- predict(model, new_data)
> pred_new
[1] no no
Levels: no yes
```

Y creando un escenario para predicción con los datos del ejemplo que hemos manejado en todos los casos, su predicción es que, en los niños de 7 y 8 años, ninguno de ellos es hablante nativo.

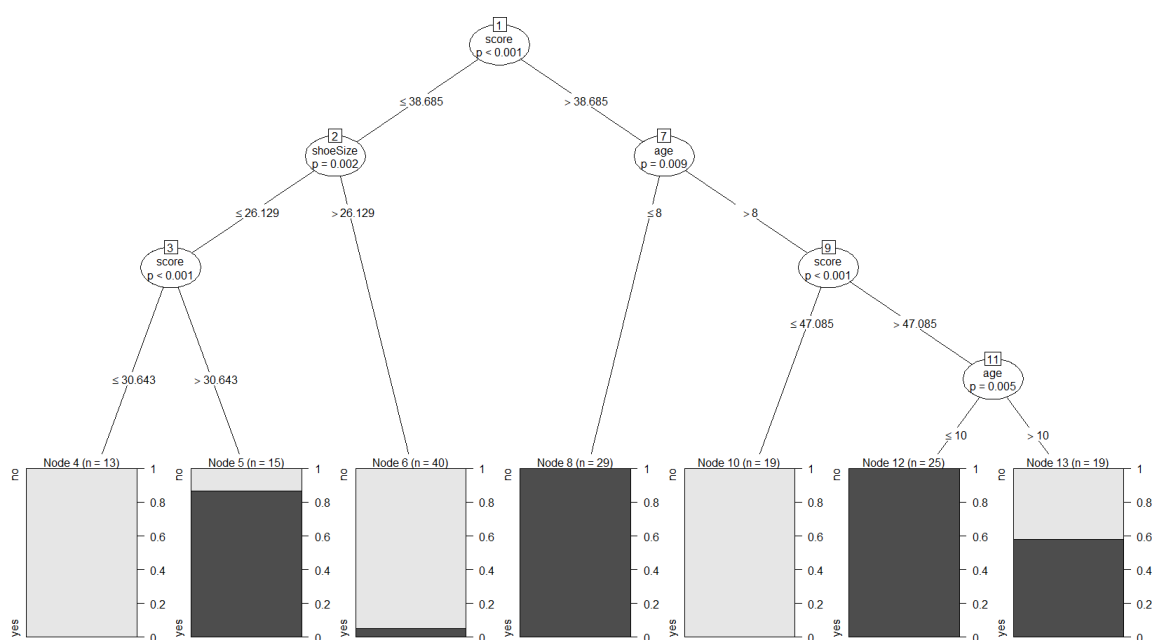
Cambiando el escenario de predicción

Para este ejercicio utilizamos el *SplitRatio* original de 0.8, la misma semilla de generación (Mi matrícula) y los datos parecen indicar equilibrio *óptimo local*.

```
> m_at
  predict_model
    no yes
no  18  2
yes  1 19
```

En la matriz de confusión durante el entrenamiento vemos uno o dos errores. Pero en este caso, lo que nos importa es cambiar la variables de:

- Edad: Ahora los casos de predicción son casos de 7 y 12 años de edad.
- shoeSize: Se quedó igual, no fue modificada y conservamos los valores iniciales de 35 y 37 respectivamente
- score: Los valores fueron reducidos a 10 y 15.



Observamos un árbol de decisiones similar al que obtuvimos cuando utilizamos un *SplitRatio* de 90%. Esto se debe a que la calidad de balance es similar, también pudo aprender suficientes reglas para crear suficientes ramas.

```
> pred_new <- predict(model, new_data)
> pred_new
[1] no no
```

Sin embargo, el resultado de la predicción es el mismo que en los experimentos anteriores. Lo que indica que la relación entre las variables específicas que decidimos cambiar no es significativa para alterar la predicción.

Conclusión del análisis del ejercicio de CTree

Al experimentar con el parámetro SplitRatio, se observó que la proporción de división de datos impacta directamente en la estructura del árbol y en la fiabilidad de las predicciones.

Con un SplitRatio bajo (0.6), el algoritmo (ctree) generó un árbol con menos nodos debido a la falta de datos de entrenamiento, resultando en un modelo generalizado con una precisión moderada (66.25%). Por el contrario, un SplitRatio alto (0.9) produjo un árbol mucho más complejo y ramificado, logrando una precisión del 90%; sin embargo, esta métrica es engañosa, ya que el conjunto de prueba se redujo a muy pocos casos, aumentando el riesgo de sobreajuste (overfitting) y haciendo que la evaluación pierda representatividad estadística.

Se concluye que el SplitRatio de 0.8 ofrece el mejor balance para este conjunto de datos, permitiendo al modelo abstraer reglas precisas (alcanzando un 92.5% de precisión) mientras mantiene un conjunto de validación lo suficientemente grande para confirmar que el modelo no se limitó a memorizar la información.

Rpart

Cambiando minsplit en rpart.control a 100

En nuestro script de rpart, la aleatoriedad viene de la función sample.split() que divide los datos en entrenamiento y prueba. Para eliminar el factor de aleatoriedad y poder hacer experimentos comparables al cambiar minsplit y cp, es necesario fijar una semilla. Nuevamente, mi *matrícula*.

```
> # Fijar la semilla para reproducibilidad  
> set.seed(1924910) # Mi matrícula 🏆
```

Ahora, creamos el modelo, yo mantendré todos los demás valores que se usaron en el script de practica pero voy a cambiar el *minsplit* de 20 a 100.

Esto en términos prácticos es decir *"No hagas una nueva rama a menos que tengas al menos 100 casas en ese grupo"*.

```
datos <- read.csv(file.choose(), stringsAsFactors = FALSE)  
  
formula_modelo <- precio ~ .  
muestra <- sample.split(datos$precio, SplitRatio = 0.8)  
train <- subset(datos, muestra)  
test <- subset(datos, !muestra)  
  
modelo_arbol <- rpart(  
  formula_modelo,  
  data = train,  
  method = "anova",  
  control = rpart.control(  
    minsplit = 100,  
    cp = 0.001  
  )  
)
```

Y esperaríamos que de un valor más grande tuviéramos un árbol más simple.

Para el resto de las variables, en específico el coeficiente de poda se mantiene igual y los valores obtenidos son los siguientes:

```
> cat("\n=== Tabla de complejidad (printcp) ===\n")

=== Tabla de complejidad (printcp) ===
> printcp(modelo_arbol)

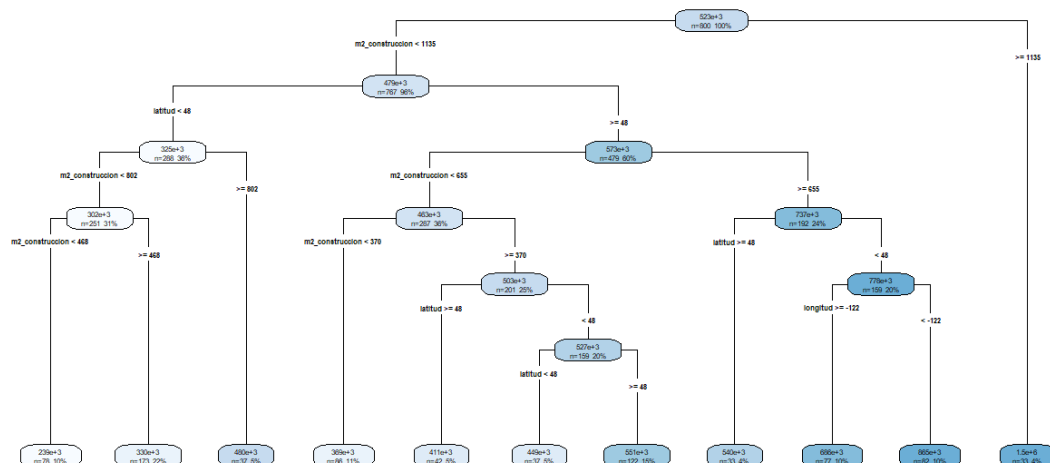
Regression tree:
rpart(formula = formula_modelo, data = train, method = "anova",
      control = rpart.control(minsplit = 100, cp = 0.001))

variables actually used in tree construction:
[1] latitud          longitud          m2_construccion m2_terreno

Root node error: 9.296e+13/800 = 1.162e+11

n= 800
```

	CP	nsplit	rel error	xerror	xstd
1	0.3793338	0	1.00000	1.00155	0.170662
2	0.1190027	1	0.62067	0.67453	0.088596
3	0.0930682	2	0.50166	0.57706	0.087328
4	0.0166753	3	0.40860	0.48386	0.086404
5	0.0136812	4	0.39192	0.47718	0.086329
6	0.0115214	5	0.37824	0.47051	0.086179
7	0.0110656	6	0.36672	0.46566	0.086100
8	0.0048342	7	0.35565	0.44685	0.086054
9	0.0047312	8	0.35082	0.44784	0.086097
10	0.0031610	9	0.34609	0.43967	0.086089
11	0.0027058	10	0.34293	0.43905	0.086067
12	0.0020265	11	0.34022	0.44094	0.086121
13	0.0010000	12	0.33819	0.44017	0.086120



En este caso, identifiqué que las variables importantes para los cálculos de correlación eran los que aparecen en la impresión.

En este caso, el modelo decidió que la ubicación (lat/long) y el tamaño (m2) eran lo único importante.

- **Root node error (1.162e+11):** Es el error inicial.
- **CP (Complexity Parameter):** Es la penalización por hacer el árbol más grande.
- **nsplit:** El número de divisiones. 0 es el tronco, 12 es el máximo de ramas que alcanzó.
- **rel error:** El error dentro de los datos de entrenamiento.
- **xerror (Cross-validation error):** Este parece ser el más importante. Es el error estimado del modelo con datos que no conoce. El xerror baja de **1.001** (fila 1) hasta **0.439** (fila 11) y luego empieza a subir un poco de nuevo (**0.440**). Ese punto donde deja de bajar es donde el modelo ya no está aprendiendo, sino memorizando.

Después se hace una búsqueda automática A donde el xerror sea el más pequeño de todos y busca qué valor de CP tiene esa fila.

```
> cp_opt <- modelo_arbol$cptable[which.min(modelo_arbol$cptable[, "xerror"]), "CP"]
> cat("\nMejor cp sugerido:", cp_opt, "\n")
```

```
Mejor cp sugerido: 0.002705828
```

El valor fue 0.002705828. Es el punto exacto donde el árbol es lo más preciso posible sin ser innecesariamente complejo.

Finalmente se toma el modelo original pero se excluyen las ramas que no ayudan lo suficiente a alcanzar el CP óptimo calculado.

```
> modelo_podado <- prune(modelo_arbol, cp = cp_opt)
>
> # Mostrar el árbol podado
> rpart.plot(modelo_podado, type = 4, extra = 101)
>
> pred_test <- predict(modelo_podado, newdata = test)
>
> RMSE <- sqrt(mean((test$precio - pred_test)^2))
> MAE <- mean(abs(test$precio - pred_test))
>
> SST <- sum( (test$precio - mean(test$precio))^2 )
> SSE <- sum( (test$precio - pred_test)^2 )
> R2 <- 1 - SSE/SST
>
> cat(sprintf("\n=== Métricas en TEST ===\nRMSE: %.4f\nMAE: %.4f\nR²: %.4f\n", RMSE, MAE, R2))
```

```
=== Métricas en TEST ===
RMSE: 177159.0700
MAE: 119726.2458
R²: 0.5757
```

RMSE (Root Mean Square Error): Es la raíz del error cuadrático medio. Mide la distancia promedio entre el valor real y la predicción.

MAE (Mean Absolute Error): Es el error promedio simple. En este caso dio **119,726**. Significa que, en promedio, el modelo se equivoca por 119 mil pesos/dólares por cada casa.

El modelo explica el **57.57%** de por qué las casas valen lo que valen. El otro 42% depende de variables que no quedaron incluidas en el árbol o que el árbol no pudo captar con un minsplit tan alto.

Cambiando minsplit en rpart.control a 5

La semilla generada sigue siendo la misma para mantener una coherencia con los datos.

```
# Fijar la semilla para reproducibilidad
set.seed(1924910) # Mi matrícula 🏆

datos <- read.csv(file.choose(), stringsAsFactors = FALSE)

formula_modelo <- precio ~ .
muestra <- sample.split(datos$precio, SplitRatio = 0.8)
train <- subset(datos, muestra)
test <- subset(datos, !muestra)
```

Pero ahora vamos a fijar el minsplit a 5

```
modelo_arbol <- rpart(
  formula_modelo,
  data = train,
  method = "anova",
  control = rpart.control(
    minsplit = 5,
    cp = 0.001
  )
)
```

Es decir, ahora estamos creando un árbol donde incluye ramas hasta para casos donde se repite un patrón incluso 5 veces. Y la mejora fue bastante notoria.

```
variables actually used in tree construction:
[1] agente_id      ano_construccion  ano_renovacion    banos
[5] codigo_frente_agua codigo_postal      codigo_vista      latitud
[9] longitud        m2_construccion   m2_terreno        pais_id
[13] pisos

Root node error: 9.296e+13/800 = 1.162e+11
```

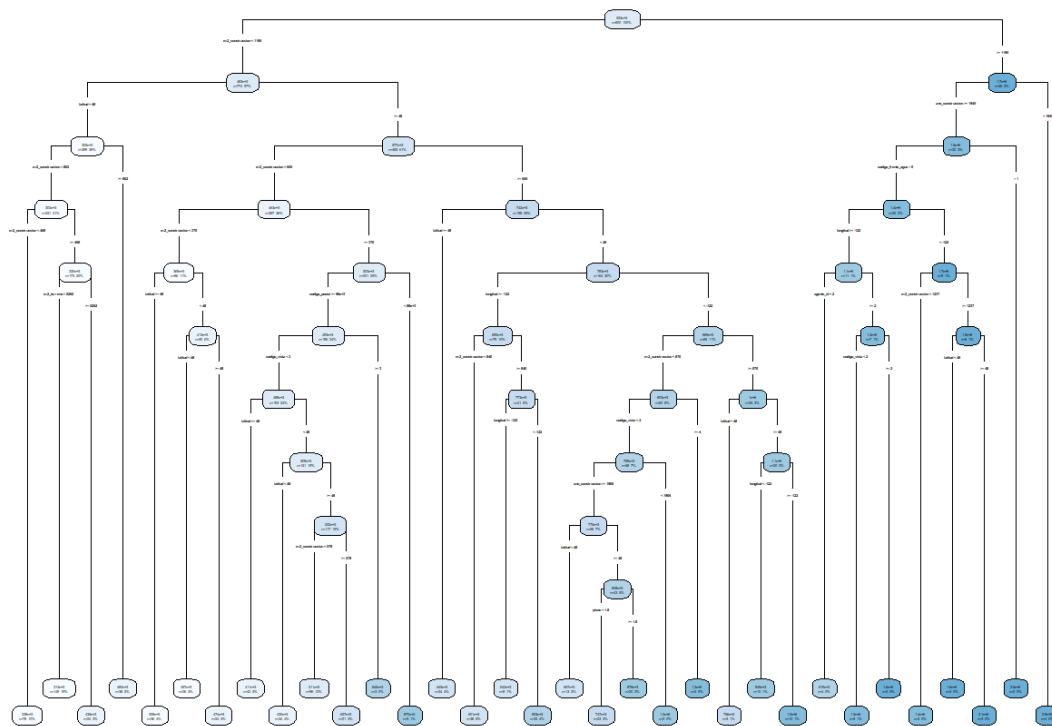
Al permitirle hacer divisiones más pequeñas, el árbol pudo usar 13 variables distintas (como año de renovación, código postal, frente al agua, etc.). Aunque en teoría un minsplit tan bajo puede causar *sobreajuste* (memorizar los datos), la función *prune* al final actuó como un amortiguador, cortando la "basura" y dejando un árbol profundo, pero altamente efectivo.

```
cp_opt <- modelo_arbol$cptable[which.min(modelo_arbol$cptable[, "xerror"]), "CP"]
cat("\nMejor cp sugerido:", cp_opt, "\n")

ejor cp sugerido: 0.001802813

modelo_podado <- prune(modelo_arbol, cp = cp_opt)

# Mostrar el árbol podado
rpart.plot(modelo_podado, type = 4, extra = 101)
```



```
> pred_test <- predict(modelo_podado, newdata = test)
>
> RMSE <- sqrt(mean((test$precio - pred_test)^2))
> MAE <- mean(abs(test$precio - pred_test))
>
> SST <- sum( (test$precio - mean(test$precio))^2 )
> SSE <- sum( (test$precio - pred_test)^2 )
> R2 <- 1 - SSE/SST
>
> cat(sprintf("\n=== Métricas en TEST ===\nRMSE: %.4f\nMAE: %.4f\nR²: %.4f\n", RMSE, MAE, R2))

=== Métricas en TEST ===
RMSE: 134779.1706
MAE: 101009.0698
R²: 0.7544
```

Obtuvimos una mejora fuertemente notable. El R^2 saltó al 75.4% y nuestro margen de error (MAE) bajó casi 19,000 unidades.

Cambiando cp en $rpart.control$ a 0.1

Para esta modificación vamos a regresar el *minsplit* a su valor predeterminado de 20 unidades, y lo que modificaremos de manera drástica es el valor en *cp*

```

· # Fijar la semilla para reproducibilidad
· set.seed(1924910) # Mi matrícula 🏆
·
· datos <- read.csv(file.choose(), stringsAsFactors = FALSE)
·
· formula_modelo <- precio ~ .
· muestra <- sample.split(datos$precio, splitRatio = 0.8)
· train <- subset(datos, muestra)
· test <- subset(datos, !muestra)
·
·
· modelo_arbol <- rpart(
·   formula_modelo,
·   data = train,
·   method = "anova",
·   control = rpart.control(
·     minsplit = 20,
·     cp = 0.1
·   )
· )

```

cp (Complexity Parameter) fue establecido en 0.1. Al ponerlo en 0.1, elegimos sólo hacer una rama si es que dicha rama mejora la precisión del modelo en un 10% de golpe.

```

> cat("\n=== Tabla de complejidad (printcp) ===\n")

=== Tabla de complejidad (printcp) ===
> printcp(modelo_arbol)

Regression tree:
rpart(formula = formula_modelo, data = train, method = "anova",
      control = rpart.control(minsplit = 20, cp = 0.1))

Variables actually used in tree construction:
[1] latitud          m2_construccion

Root node error: 9.296e+13/800 = 1.162e+11

n= 800

      CP nsplit rel error  xerror   xstd
1 0.41470     0  1.00000 1.00155 0.170662
2 0.12338     1  0.58530 0.61509 0.067199
3 0.10000     2  0.46192 0.51095 0.064787
>

```

Nos dejó solamente con dos variables. *Latitud* y *m2 de construcción*.

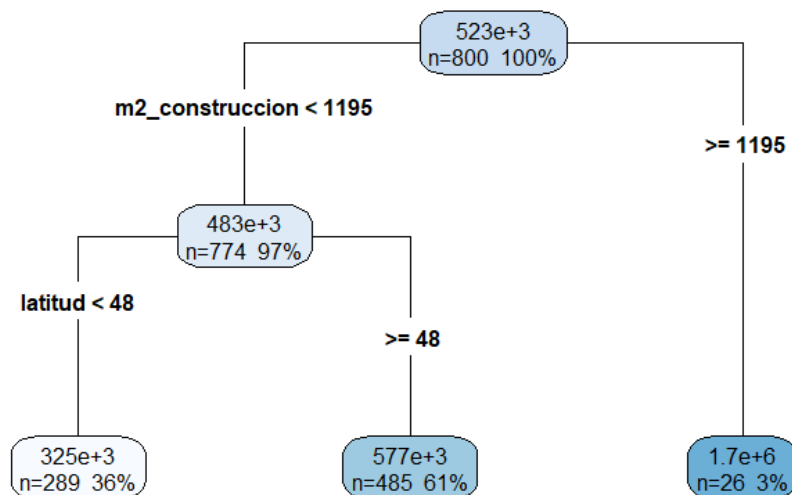
```

> cp_opt <- modelo_arbol$cpstable[which.min(modelo_arbol$cpstable[, "xerror"]), "cp"]
> cat("\nMejor cp sugerido:", cp_opt, "\n")

Mejor cp sugerido: 0.1
>
> modelo_podado <- prune(modelo_arbol, cp = cp_opt)
>
> # Mostrar el árbol podado
> rpart.plot(modelo_podado, type = 4, extra = 101)
>
> pred_test <- predict(modelo_podado, newdata = test)
>
> RMSE <- sqrt(mean((test$precio - pred_test)^2))
> MAE <- mean(abs(test$precio - pred_test))
>
> SST <- sum( (test$precio - mean(test$precio))^2 )
> SSE <- sum( (test$precio - pred_test)^2 )
> R2 <- 1 - SSE/SST
>
> cat(sprintf("\n=== Métricas en TEST ===\nRMSE: %.4f\nMAE: %.4f\nR²: %.4f\n", RMSE, MAE, R2))

=== Métricas en TEST ===
RMSE: 238599.9350
MAE: 170790.4677
R²: 0.2304

```



Un desastre predictivo. El R^2 se desplomó al 23% y el error se disparó a 170,790. El árbol solo usó 2 variables porque ninguna otra variable lograba ese estricto 10% de mejora requerido.

Esto es un caso de manual de **Subajuste (Underfitting)**. Le pusimos reglas tan estrictas que el modelo no pudo aprender casi nada del DataSet.

Conclusión del análisis del ejercicio de Rpart

Parámetro Modificado	MAE (Error Promedio)	R ² (Precisión Global)	Variables Usadas
minsplit = 100	119,726.24	0.5757 (57.5%)	4
minsplit = 5	101,009.06	0.7544 (75.4%)	13
cp = 0.1	170,790.46	0.2304 (23.0%)	2

El análisis del modelo de regresión (rpart) para la predicción de precios de viviendas demuestra la importancia crítica de la sintonización de hiperparámetros.

Se comprobó que un parámetro de complejidad (cp) demasiado alto, como 0.1, genera un subajuste severo, limitando el árbol a solo dos variables y reduciendo la varianza explicada (R^2) a un deficiente 23%. > Por otro lado, la modificación del número mínimo de observaciones para dividir un nodo (minsplit) mostró que el conjunto de datos contiene relaciones complejas que requieren profundidad para ser modeladas. Mientras que un minsplit de 100 entregó un modelo generalizado con un error medio absoluto (MAE) de 119,726, reducir el minsplit a 5 permitió al árbol capturar sutilezas en 13 variables distintas (como código postal y características físicas). Esta flexibilidad, combinada con una poda automatizada mediante el CP óptimo (0.0018), resultó ser la configuración superior, elevando el R^2 a 75.44% y reduciendo el MAE a 101,009. Se concluye que, para este DataSet inmobiliario, permitir una topología de árbol más profunda arroja predicciones significativamente más robustas y precisas.

Conclusiones generales

La presente práctica resultó ser una de las más extensas y demandantes del curso (Hasta ahora..., pero también fue la que permitió comprender con mayor claridad los principios fundamentales de la minería de datos. A lo largo de las actividades realizadas, se trabajó con conjuntos de datos que, en apariencia, consisten únicamente en tablas de valores numéricos y etiquetas, pero que en realidad contienen relaciones matemáticas complejas que no son evidentes a simple vista. La práctica demostró que estas relaciones pueden ser descubiertas mediante el uso de modelos computacionales que exploran distintas configuraciones y evalúan su desempeño, evidenciando el carácter experimental y iterativo del análisis de datos.

Uno de los aspectos más relevantes fue el estudio de los árboles de decisión basados en entropía. Antes de realizar los cálculos, resultaba poco intuitivo (para mí) pensar que el orden en que se utilizan las mismas variables pudiera afectar la cantidad de información obtenida.

Conceptualmente, podría esperarse que la interacción entre tres variables contenga siempre la misma información total, sin importar cómo se estructure. Sin embargo, los resultados mostraron que la forma en que se organiza el árbol influye en la reducción progresiva de la incertidumbre, lo que pone de manifiesto la importancia de la secuencia en la que se aplican las particiones y cómo esta afecta la eficiencia del modelo. Me gustaría estudiar a nivel teórico más este fenómeno porque de alguna manera logré replicarlo de forma experimental sin entender las bases lógicas de este suceso.

Asimismo, la práctica permitió diferenciar con mayor claridad entre distintos tipos de árboles utilizados en minería de datos. Inicialmente se asumió que los árboles binarios y los árboles de decisión eran equivalentes, debido a la similitud en su estructura, e ingenuamente confiando también en la similitud de su nombre “árbol”. Sin embargo, al trabajar con herramientas como **ctree** y **rpart**, se observó que existen diferencias importantes tanto en su propósito como en su funcionamiento. Mientras que **ctree** se emplea principalmente para tareas de clasificación, donde el objetivo es predecir categorías, **rpart** puede utilizarse para problemas de regresión, en los que se busca estimar valores numéricos continuos. Esta distinción permitió comprender que la estructura de árbol es solo una representación general, y que su comportamiento depende del algoritmo y del tipo de problema que se esté abordando.

Finalmente, uno de los aspectos más impactantes de la práctica fue observar cómo un modelo computacional puede realizar predicciones numéricas a partir de patrones detectados en los datos. El hecho de que un algoritmo sea capaz de estimar un valor continuo, como el precio de una vivienda, basándose únicamente en la relación entre múltiples variables, pone en evidencia la capacidad de los métodos de minería de datos para extraer conocimiento útil a partir de grandes volúmenes de información. Este proceso, aunque inicialmente puede parecer poco intuitivo, se fundamenta en principios matemáticos sólidos y demuestra el potencial de estas técnicas en aplicaciones reales.

Aunque algunos resultados no fueron completamente intuitivos en un primer momento, el análisis posterior ayudó a reconocer que estas diferencias reflejan propiedades reales de la información y de la estructura de los datos, lo cual constituye uno de los aprendizajes más valiosos obtenidos durante el desarrollo de la actividad.

Referencias

- [1]. QUINLAN, J. R. 1986. INDUCTION OF DECISION TREES. MACH. LEARN. 1, 1 (MAR. 1986), 81–106
- [2]. HAN, J., KAMBER, M., & PEI, J. (2012). DATA MINING: CONCEPTS AND TECHNIQUES. ELSEVIER.
- [3]. NOWOZIN, SEBASTION (2012-06-18). "IMPROVED INFORMATION GAIN ESTIMATES FOR DECISION TREE INDUCTION". ARXIV:1206.4620V1 [CS.LG].
- [4]. QUINLAN, J. R. C4.5: PROGRAMS FOR MACHINE LEARNING. MORGAN KAUFMANN PUBLISHERS, 1993.
- [5]. SHANNON, C. E. (1948). A MATHEMATICAL THEORY OF COMMUNICATION. BELL SYSTEM TECHNICAL JOURNAL.